

Contenido

Git y Github.....	2
Enlaces para instalar	2
¿Qué es GIT?	2
Flujo de trabajo de GIT	2
¿Qué es GitHub?	2
Comandos básicos	3
Trucos.....	3
Diferencias entre -- y -.....	3
Creando alias globales	4
Renombrar archivos.....	4
Eliminar archivos	4
Ignorando Archivos	4
Ramas o branch	5
Atajos	5
Tags	5

Git y Github

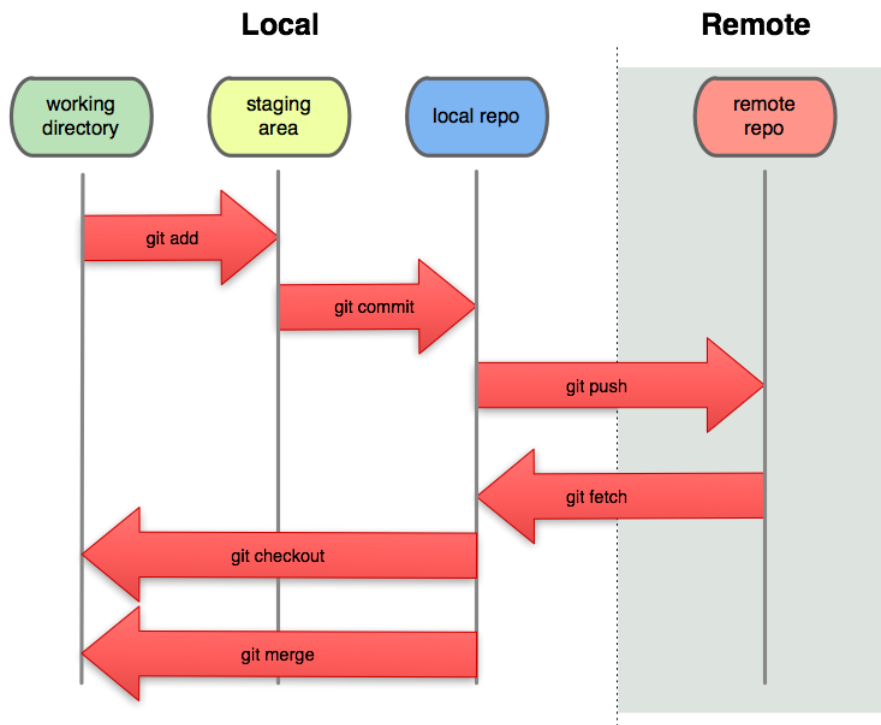
Enlaces para instalar

- ✓ [Instalar Git](#)
- ✓ [Github](#)

¿Qué es GIT?

Es un software de control de versiones, su propósito es llevar registro de los cambios en archivos de computadora y coordinar el trabajo que varias personas realizan sobre archivos compartidos (También puedes trabajar solo no hay problema 😊). Existe la posibilidad de trabajar de forma remota y una opción es GitHub.

Flujo de trabajo de GIT



¿Qué es GitHub?

Es una plataforma de desarrollo colaborativo para alojar proyectos (en la nube) utilizando el sistema de control de versiones Git. Además cuenta con una herramienta muy útil que es GitHub Pages donde podemos publicar nuestros proyectos estáticos (HTML, CSS y JS) gratis.

Comandos básicos

Aprendamos los primeros comandos con git

- ✓ Versión de git

```
git version
```

- ✓ Registrar nuevo usuario asociado a git:

```
git config --global user.name "usuario"
```

```
git config --global user.email "usuario@gmail.com"
```

- ✓ Ayuda

```
git help
```

- ✓ Mi primer repositorio

```
// Iniciar un nuevo repositorio
```

```
// Crear la carpeta oculta .git
```

```
git init
```

```
// Ver que archivos no han sido registrados
```

```
git status
```

```
// Agregar todos los archivos para que esté pendiente de los cambios
```

```
git add .
```

```
// Crear commit (fotografía del proyecto en ese momento)
```

```
git commit -m "primer commit"
```

```
// Muestra la lista de commit del más reciente al más antiguo
```

```
git log
```

En resumidas cuentas, nosotros realizamos cambios en nuestros archivos, el comando `status` verificará que archivos han sido modificados. Cuando deseemos registrar esos cambios tendremos que agregarlos con `add` . así ya estará listo para poder hacer un commit. El `commit` realiza la copia de ese instante para poder volver en el tiempo si es que es necesario.

Trucos

```
// Muestra en una línea los commit realizados
```

```
git log --oneline
```

```
// Muestra en una línea los commit realizados pero más elegante
```

```
git log --oneline --decorate --all --graph
```

```
// Solo muestra los archivos modificados
```

```
git status -s
```

Diferencias entre -- y -

- ✓ `--decorate` hace referencia a una palabra
- ✓ `-s` hace referencia al comando o a varios comandos, `-sa` serían dos comandos diferentes

```
// Vemos información de la rama maestra
```

```
git status -s -b
```

```
//Hace lo mismo que el comando anterior
```

```
git status -sb
```

Creando alias globales

Los alias nos sirven para crear atajos de comandos, podemos guardar diferentes alias de forma global y quedarán guardados en la configuración de git.

```
// Guardamos el alias "lg" que ejecutará todo lo que está entre comillas
```

```
git config --global alias.lg "log --oneline --decorate --all --graph"
```

```
// Para ver el archivo config con los alias creados
```

```
git config --global -e
```

```
git commit --amend
```

Renombrar archivos

Puede que queramos renombrar un archivo, es recomendable hacerlo directamente en la línea de comandos para registrar los cambios con git.

```
// Cambiar nombre
```

```
git mv nombreOriginal.txt nombreNuevo.txt
```

Recuerden hacer el commit para registrar los cambios en git.

Eliminar archivos

```
// Cambiar nombre
```

```
git rm nombreArchivo.vue
```

También recordar hacer el commit para salvar cambios en git.

Ignorando Archivos

Para no hacer seguimiento de carpetas o archivos, debemos crear el siguiente archivo `.gitignore` Su contenido sería así:

```
archivo.js // Ignora el archivo en cuestion
```

```
*.js // Ignora todos los archivos con extensión .js
```

```
node_modules/ //Ignora toda la carpeta
```

Ramas o branch

Hasta el momento solo hemos trabajado en la rama "master" pero puede que necesitemos crear diferentes ramas para los seguimientos de git.

```
// Crea una nueva rama
git branch nombreRama

// Nos muestra en que rama estamos
git branch

// Nos movemos a la nueva rama
git checkout nombreRama
```

Podemos unir la rama master con la nueva, para eso tenemos que estar en la master para ejecutar el siguiente comando:

```
// Nos movemos a la nueva rama
git merge nombreRama

// Eliminar una rama
git branch -d nombreRama
```

Atajos

Podemos utilizar `git checkout -b nuevaRama` para crear la nuevaRama y viajar a ella.

Tags

Con los tags podemos hacer versiones de nuestro proyecto.

```
// Crear un tags
git tag versionAlpha -m "versión alpha"

// Listar tags
git tag

// Borrar tags
git tag -d nombreTags

// Hacer una versión en un commit anterior ej: f52f3da
git tag -a nombreTag f52f3da -m "version alpha"

// Mostrar información del tag
git show nombreTag
```